

智能纪要：Development Work Discussion

2026年7月23日

🕒 7月23日(周四) 10:03 - 12:22 (GMT+08)

📍 雅聚阁佛缘堂

纪要 原始记录

总结

Development Workflow and Registration Priorities Discussion

- 💡 **Key Decisions:**
- Development will be aligned to the main branch with **Clement** as the primary PR approver to ensure framework consistency.
 - Michael's registration demo** is the immediate priority, requiring completion of the UI form and integration with the workflow service.
 - The team will adopt a structured approach for event modeling and API validation to ensure clear documentation and system interoperability.

Development Workflow & Branch Strategy

📌 Branch Alignment Strategy

Key Decision

- Current release branch to be merged into **main**
- All future feature branches to be created from **main**
- Demo branches can exist in parallel but must sync
- Clement** will be the primary approver for PRs to main
- Ensures UI consistency and reduces rework

🎨 UI Development & Storybook

- All UI components must reuse existing **Storybook** components
- New UI components must be contributed back to the UI Framework team
- The Storybook will become a separate **NPM package**
- Reduces token wastage and ensures consistent user experience

Registration Module & Event Modeling

📌 Registration Demo Priority

- Michael's** registration demo is the top priority
- Registration screen resides in UI, workflow logic in separate service
- Need to add **file upload** functionality to registration form
- "Acknowledgement Work Desk" page displays request statuses
- Registration is agnostic of underlying trade product

📌 Event Modeling Approach

- Event modeling required for documenting business flows
- Event sourcing only required for processing box (e.g., guarantees)
- Reference data modules do not require event sourcing
- Team to experiment with generating event models from **BRDs/specs** using AI tools
- Reverse engineer existing code to understand data flows and APIs

📌 API & Validation Strategy

- First API is the registration API in the workflow service
- Use **Java record** with validation annotations for payloads
- Validations must be reflected in the **OpenAPI** contract
- Need script to sync API contract changes with UI schemas
- Ensures front-end and back-end validation consistency

This meeting discusses cross-team development rules, registration module progress, tooling support, and architecture alignment, with clear task arrangements and standard requirements as follows:

- **Branch Management & Cross-Team Synchronization Workflow**

- **Main Branch Initialization Arrangement**

- Release Branch Cut Plan: The existing stable release branch (tentatively named release 2024.06.04) will be first merged into the currently empty main branch as the initial baseline for all subsequent cross-team development.
- Feature Branch Naming Rule: All new feature branches must use the prefix "feature/" followed by the module name and ADO task number, and prohibit arbitrary naming to ensure traceability across the entire development team.
- Conflict Resolution Strategy: Shared logic conflicts such as authentication will be resolved by a dedicated team member in a one-time manner, avoiding repeated conflict resolution work for each individual module developer.
- Demo Branch Isolation Rule: The existing demo branch for the registration module will be retained independently, and no mandatory merge to the main branch is required before the current demo is completed to avoid unexpected operation failures.
- One-Time Merge Principle: The merge to the main branch is a one-time activity, and after the merge is completed, the temporary merge branch can be deleted, and all subsequent development will be directly based on the main branch baseline.
- Module Isolation Guarantee: The code structure of each module is independent of each other, and the development of the registration module will not affect the development progress of other modules such as the reference data module.
- Synchronization Frequency Requirement: All developers need to synchronize the latest content of the main branch at least once every two days, to avoid the accumulation of a large number of conflicts caused by long-term non-synchronization.

- **Environment Mapping & Deployment Rule**

- Local Demo Environment: Developers use local independent branches for quick verification and internal standup sync, and do not need to synchronize to the remote environment immediately after the development is completed.
- Dev Environment Mapping: The dev 0.6 environment is automatically built from the latest main branch submission, and all merged feature branches will be deployed to this environment for full test verification.

- **QA & Release Promotion:** After the dev environment test passes, the binary will be promoted to the QA 0.6 environment, and the official sprint demo will be run in the QA environment instead of the local environment.
- **PR Approval Governance:** All pull requests merged into the main branch require dedicated architecture team review and approval first, to avoid unqualified framework changes being merged into the core branch.
- **Merge Sequence Arrangement:** The merge to the main branch will be carried out module by module, and the team will not carry out the merge of all modules in a big bang manner to reduce the overall risk of conflict.
- **Conflict Resolution Support:** If developers encounter unsolvable conflicts during the merge process, they can contact the dedicated support personnel to assist in resolution, to avoid blocking the normal development progress.

Environment	Deployment Source	Usage Scenario	Approval Requirement
Local Demo	Independent local feature branch	Quick verification, internal standup sync	No approval required
Dev 0.6	Latest main branch commit	Full test after feature merge	Architecture team PR review
QA 0.6	Binary promoted from Dev 0.6	Official sprint demo, pre-release check	Module owner confirmation

All features must be merged to main module by module to reduce overall conflict risk.

AI-generated

Development Environment Mapping & Deployment Rules

Rule Item	Specific Requirement	Core Purpose
Main branch baseline	Merge release 2024.06.04 to empty main first	Unify cross-team development baseline
Feature branch naming	Prefix with 'feature/' + module name + ADO number	Ensure full traceability across team
Conflict resolution	Dedicated member resolves shared logic conflicts once	Avoid repeated work for individual developers
Sync frequency	Sync latest main content at least every 2 days	Prevent large conflict accumulation

Existing registration demo branch will be retained independently before demo completion.

AI-generated

Core Branch Management Rules

- **Registration Module Development Progress & Task Arrangement**
 - **Current Development Status Confirmation**
 - **End-to-End Test Completion:** The back-end logic of the registration module has been completed, and the full link test from UI form submission to workflow service persistence has been verified to be fully runnable.
 - **Temporal Installation Requirement:** Developers need to install a standalone Temporal service locally via `brew install temporal` , and access the Temporal dashboard via `localhost:8233` to visualize the workflow execution status.

- Data Routing Logic: The product code field submitted by the registration form will be used as the core basis for subsequent request routing, and the current test uses the ACL product type which will be adjusted to the guarantee type later.
- Attachment Function Development: The registration page needs to add a file upload component, and the uploaded file data will be associated with the corresponding registration request ID for subsequent query and review.
- Temporal Dashboard Access: The Temporal test server currently used locally is unstable, and it is recommended to install a standalone Temporal service to obtain a complete and stable workflow visualization dashboard.
- Request Tracking Mechanism: The registration request submitted by the UI will generate a unique creation ID, which is used to track the full link execution status of the request in the workflow service.
- Form Field Mapping: All fields in the registration form, including booking location and product type, will be persisted to the database as part of the request body, and will be displayed in the subsequent request status page.

○ **Subsequent Development Arrangement**

- Request Status Page Adjustment: The existing request status page will be renamed to the acknowledgement work desk, and columns including submission method and booking location will be added to match the registration form field content.
- Reference Data Reverse Engineering: The team will reverse engineer the existing mature registration module implementation, extract the data source logic of each form field to reduce unnecessary cross-team communication and query.
- X Reference Data Exclusion: The registration module development does not depend on the X reference data module for the time being, and all required data can be obtained from the existing workflow and reference data services.
- Workflow Isolation Confirmation: The workflow service corresponding to the registration module is an independent new service, which does not share a repository with the existing reference data module, and has no coupling with other existing services.
- Request Status Query: The workflow service provides a status query interface, which can query the current execution node and historical execution records of the request through the unique registration ID.
- Reverse Engineering Output: The reverse engineering of the existing registration module will output complete architecture documentation, which can be directly queried by developers when encountering problems, reducing cross-team communication costs.

- **Event Modeling & Tooling Support Specification**

- **Event Modeling Tool Usage Rule**

- **Martian Tool Automation:** The Martian tool integrated in the Borg platform can automatically generate the initial event model by importing the existing board and diagram, and only requires minor manual adjustment of individual nodes.
 - **Generation Effect Test Plan:** The team will test the effect of generating event models from BRD and specification documents respectively, and select the input source with the highest accuracy as the standard input for subsequent development.
 - **Slice Export & Reuse:** The completed event model slice can be exported as a JSON file, and attached to the corresponding ADO task as a specification to automatically generate boilerplate code for commands and events.
 - **Business Flow Documentation Requirement:** All modules need to complete the corresponding event model as the official business flow documentation, to ensure that all team members have a consistent understanding of the module logic.
 - **Martian Tool Access:** The Martian event modeling tool can be accessed through the Helix Dev platform, and the tool entry is located in the Tools drop-down menu of the official platform.
 - **Event Model Granularity Control:** The event model only needs to cover the core business nodes, and does not need to list all technical implementation details, to avoid the model being too complex to be understood by the team.
 - **Happy Path Priority:** The event model development first completes the full link of the normal happy path, and the abnormal branch logic such as rejection and timeout will be supplemented in the subsequent iteration stage.

- **Event Sourcing Implementation Scope**

- **Restricted Implementation Scope:** Only the core transaction processing module under the product processing box will implement event sourcing, and no other modules will use this architecture for the time being to avoid unnecessary complexity.
 - **Processing Module Flow Requirement:** The event model of the registration module needs to cover the full happy path from form submission to product routing, and the rejection branch logic will be supplemented in subsequent iterations.
 - **Component Reuse Requirement:** The generated event model needs to reuse existing components as much as possible, and new components need to be submitted to the storybook repository as a pull request after confirmation.
 - **Temporary Dependency Exemption:** The event model generation tool is still in the deployment and adjustment stage, and the unavailability of the tool will not block the progress of existing development tasks.

- Input Source Test Plan: The team will test the effect of generating event models from BRD and functional specifications respectively, and select the input source with the highest accuracy as the unified standard.
- Tool Dependency Exemption: The Martian event modeling tool is still in the deployment and adjustment stage, and the unavailability of the tool will not block the existing development task progress for any module.
- **UI Component Consistency & Standardization**
 - **Storybook Usage Requirement**
 - Component Reuse Mandate: All UI development must first search for reusable components in the existing storybook, and prohibit developers from independently creating custom components of the same type to avoid inconsistent user experience.
 - New Component Contribution Rule: If the required component does not exist in the storybook, the developer needs to submit the developed component to the UI framework team as a pull request for unified maintenance.
 - Future Storybook Deployment: The storybook will be separated from the existing web UI repository and released as an independent NPM package, which is convenient for all teams to introduce and use in a unified manner.
 - Existing Code Cleanup Task: The team will use the existing code cleaning skill to scan the reference data module, and replace the custom components that do not conform to the storybook specification in batches.
 - Component Version Management: After the storybook is released as an independent NPM package, the version number will be managed uniformly, and all teams will use the specified version to avoid the problem of inconsistent component versions.
 - UI Framework Team Responsibility: The UI framework team is responsible for the unified maintenance of the storybook, and all new components need to be reviewed by the team before being merged into the storybook repository.
 - **Figma Replacement Arrangement**
 - Figma Access Solution: The Figma platform will be gradually retired due to high cost, and subsequent UI design will no longer rely on Figma, and wireframe level structure description can meet the development requirements.
 - Style Specification Source: The UI style specification will be obtained from the Figma MCP server, and developers can directly align the style in the local branch according to the specification without accessing the Figma platform.
 - UI Consistency Check: The UI auditor skill will be introduced in the subsequent pull request review process, to automatically check whether the components used in the submitted code conform to the unified specification.

- User Experience Consistency Requirement: All pages of the product must maintain a consistent interaction and display style, and the difference in component style between different modules is strictly prohibited to avoid user confusion.
- Figma Retirement Arrangement: The Figma platform will be gradually retired due to high cost, and subsequent UI design only needs wireframe level structure description to meet the basic development requirements.
- Figma MCP Server Usage: Developers can directly obtain the UI style specification from the Figma MCP server, and do not need to apply for a separate Figma account or verify the email to complete the style alignment.
- **Validation & API Specification Requirements**
 - **Validation Layer Implementation Rule**
 - Payload Isolation Principle: The data payload is defined as a pure Java record, and the validation logic is independently implemented in a separate interface layer, which is uniformly called by the Mali framework to trigger validation.
 - Validation Consistency Requirement: The same set of validation rules will be used at both the front-end UI layer and the back-end API layer, to avoid inconsistent validation results caused by separate implementation of rules at both ends.
 - Open API Synchronization: The validation rules defined in the back-end interface will be automatically synchronized to the Open API document, and the field validation description can be directly viewed in the interface document.
 - UI Schema Synchronization: The team will develop an automatic synchronization script, which will automatically update the front-end validation schema after the back-end API specification changes, to avoid out of sync between the two ends.
 - Multi-Region Deployment Guarantee: As long as the UI and Postgres service are running normally, users can submit registration requests normally even if the Temporal service is completely down.
 - Kafka Usage Specification: Kafka is only used for asynchronous event notification, and is not used for synchronous request response, to avoid tight coupling between independent services.
 - **Subsequent Verification Arrangement**
 - First Verification Object: The first batch of verification of the validation framework will be carried out on the registration back-end API, to confirm that the validation rules and Open API document generation effect meet expectations.
 - Validation Library Confirmation: The team will confirm the adaptation of the existing validation library and annotation composition scheme, and form a unified development specification for subsequent modules to use.

- Audit Trail Requirement: All validation execution results will retain a complete audit trail, which is convenient for subsequent problem location and compliance review.
- Multi-Region Deployment Adaptation: The architecture design ensures that as long as the UI and Postgres service are running normally, the user can submit the registration request normally even if the Temporal service is down.
- API Contract First Principle: All new interfaces must first complete the Open API specification definition, and then carry out back-end and front-end development according to the specification to ensure the consistency of the interface.
- Validation Rule Extension: The validation framework supports custom validation rules, and developers can extend new validation rules according to business needs, and uniformly register them in the Mali framework.

👉 智能纪要反馈收集 [该内容不支持导出查看]

智能章节

00:00 Work Plan Clarification on UI Development Branch Arrangement and Event Modeling Task

This section covers two core work arrangements. First, confirm with Clement the dedicated UI trunk branch to carry out trunk-based development, require at least two PR submissions daily before and after lunch, and later align front and back end work to the main branch after sorting out the current branch mess. Second, assign Ziming to work on event modeling, which can be largely auto-generated via the Martian tool from existing BRD board content and provided diagrams with minimal manual adjustments.

04:07 Discussion on Workflow Application, Module Development and Project Progress Confirmation

This meeting chapter discusses development-related arrangements: the team will not create a new X-song ripple, instead use product service EX's existing module or build a new sub-module inside it. The workflow deployment location needs further confirmation with Lups. Pending tasks include confirming the usable branch with Clement, after which Harry will takeover to launch the relevant branch, and formal UI development can start then.

08:11 Discussion on Team Work Priorities and Task Assignment

This meeting section focuses on work priority alignment. The top priority is completing the registration module, with clarified split responsibilities: Michael takes charge of its workflow service, while assigned members handle the supporting registration service. The next confirmed priority is the transaction working desk, which should be launched soon to avoid

pressure from Nox and business teams. The team also agrees to sort out and record all members' specific assigned work parts after the call for clear reference.

12:03 Discussion on Registration SIT Progress, Temporal Deployment and UI Adjustment Requirements

This meeting chapter covers progress on the registration workflow system: the backend setup for end-to-end SIT testing is completed, using creation ID to trace requests from UI to workflow service. The outbox pattern and multi-AZ AWS cross-region deployment logic are clarified. Michael will install local Temporal for workflow visualization, adjust product types to connect the registration service, and modify the request status page to add required fields to align with the baseline UI.

38:00 UI Development Workflow and Component Consensus Discussion

This chapter clarified UI development specifications: all members shall work off main branch, reuse Storybook components to cut token waste, ensure UI consistency. Figma will be phased out, missing new components need PR to UI Framework team, Storybook will be a separate versioned NPM package. Teams will test the AI skill to align existing inconsistent UI, use reverse-engineering on existing modules to extract architecture docs, run quick experiments to pick the most effective way to generate proper event models.

57:45 Discussion on Git Branch Alignment, Merging Strategy and Demo Arrangements

This meeting section focuses on Git branch alignment and release arrangement: they note current separate demo and release branches have inconsistent strategies, decide to run upcoming demos on existing stable demo branches while keeping demo branches untouched to avoid disruption. They set a clear branch hierarchy of main, intermediate team branch, then Genesis team, plan to name the release branch formally, and arrange parallel priority-adjusted sync work to eventually merge all valid changes to main.

01:09:08 Git Branch Naming and Merging Workflow Discussion

This chapter discusses the team's current branch management setup: all developers build their own feature branches based on a shared branch, which can be renamed properly for clarity to support demo needs. The proposed phased workflow first cuts and synchronizes the release branch, then merges code to main incrementally instead of a big-bang approach, to avoid disrupting demos while keeping cross-branch sync until full code completion.

01:15:27 Walkthrough of Multi-Environment Deployment Pipeline Workflow

This section walks through the team's multi-environment deployment flow: the allocated Dev06 EKS environment is automatically built every time code is pushed to main, then flows to QA6 with no further builds after that. The finalized binary will be promoted sequentially to OPD and staging, with no questions raised by attendees after the walkthrough.

01:19:34 Discussion on Harry Fisher's workflow of merging his work to main

This meeting section centers on the merge process for Harry's work covering Feature Harry 1, 2, 3 and related Harry Fisher modules. Speaker 2 raises the question of whether Harry will follow the same process as merging feature coded decoder to main for his work, and Speaker 3 responds confirming the applicable merge arrangement for Harry's involved modules, after which Speaker 2 confirms he has got the clear related information.

01:21:22 Discussion on Git Workflow, Merge Conflict Resolution and PR Governance Rules

This chapter discusses code collaboration workflows: it notes the authentication module has shared changes to avoid redundant conflict resolution, allows flexible task handling per personal work style, clarifies local demos are for quick syncs while official demos are on 20260.4 QA environment, designates Clement as primary PR approver for initial governance to avoid bottlenecks later, requires short single-purpose feature branches, and confirms follow-up points including upcoming feature sizing and shared environment docs.

01:30:48 Discussion on Task Priorities and Related Technical Arrangements for Michael's Registration Demo

This meeting section confirms top two priorities: Michael's registration demo and code synchronization. It clarifies the two tasks can run in parallel with minimal code conflicts, as long as no one overrides Michael's work. It also aligns rules for event modeling and event sourcing: event modeling is just for business flow documentation, no event sourcing is applied to reference data, only the current transaction processing box will involve action dependency, and the registration demo does not need to include Xon.

01:36:53 Discussion on Microservice Architecture Rules and Module Division for Financial Processing System

The meeting first resolved screen sharing issues for a nearly 8-year-old architecture diagram, then confirmed separate independent service modules including UI, reference data, workflow and processing (which covers supplier finance and other product lines). It defined cross-module communication rules: REST APIs for interaction, Kafka only for unidirectional notifications that do not expect responses, noting the aspect-driven development kit helps enforce these architecture constraints.

01:41:31 Discussion on Box Color Meaning and Registration Related Confirmation

This meeting section focuses on two core topics. First, the participants clarify the background color coding of boxes in relevant diagrams: the green boxes represent ODS integration reference data, the rest are blue, and the color distinction actually has no special practical meaning, which was just set for a better presentation effect before. Second, the two sides confirm the current progress of the registration work, noting that the page has been populated with workflow data and reference data while there is no processing box yet, and the team needs to start the formal registration part subsequently.

01:43:38 Event Modeling Practice and Registration Flow Design for Trade Business Systems

This session clarifies that event sourcing is only for processing while event modeling documents business flows, then collaboratively drafts a generic trade-agnostic registration standalone bounded context event model covering happy path, rejection logic, related views, routers, and subsequent guarantee application flows including compliance checks. The team also introduces the event modeler tool, confirms shared access to the drafted diagram, and wraps up after verifying no further questions.

02:11:41 Discussion on Validation Library Implementation and OpenAPI-based API Validation Synchronization

This section centers on validation solution discussions: the team confirmed the validation library, opted to keep data as pure Java records with all validations centralized in one place. They agreed to ensure consistent validations are reflected in Open API contracts, and will first implement this validation pattern on Michael's backend Registration API to check the API spec. They also plan to develop a sync script to avoid UI and backend schema desync whenever API changes.

👉 智能纪要反馈收集 [该内容不支持导出查看]

关键决策

- **Key Decision:** The team will adopt the incremental main branch alignment process for codebase synchronization, which keeps existing demo branches unaffected while gradually migrating all feature modules to develop from the main branch.
 - **Problem:** Multiple existing feature/demo branches have inconsistent architecture and authentication logic, causing alignment conflicts that risk ongoing demo delivery.
 - **Discussion Plan:**
 - Speaker 3 proposes that the release branch first synchronizes content to the empty main branch, then each module owner creates a dedicated merge feature branch to handle conflict resolution separately without touching the original demo branch.
 - Speaker 3 notes that short, single-purpose feature branches reduce subsequent synchronization complexity, and merged content will be deployed to the Dev 6 environment for verification before entering the release pipeline.
 - Speaker 5 suggests keeping the current demo branch for near-term stakeholder demos, with the next round of demos running on the QA 6 environment after main branch integration.

- **Decision Basis:** This approach avoids disrupting ongoing demo delivery, distributes conflict resolution workload to individual module owners, and establishes a standardized long-term development workflow.
-

- **Other Decisions:**

- a. Ziming is assigned to complete the event modeling work for the registration module, using existing automation tools to generate initial drafts.
- b. All UI development must reuse components from the shared storybook to ensure consistent user experience across the entire application.
- c. Event sourcing is only applied to the transaction processing module for now, and will not be used in other modules until the team masters the fundamentals.
- d. The registration screen and related logic will be placed in the workflow service repository, with cross-service communication limited to REST APIs and Kafka notifications.
- e. The first API to be standardized with validation rules is the backend registration API, with subsequent UI validation logic aligned to the API contract.

👉 智能纪要反馈收集 [该内容不支持导出查看]

金句时刻

「It's one app. He logs into one app, but when he goes to registration, the UI looks different. The user experience is different. If he goes to BCA terms, the user experience is different.」

—— This clearly states the core requirement for UI consistency, which is the key driver for the team to adopt the shared storybook component system.

「Get out of the habit of long, deep feature branch. That way you don't, your synchronization, your life will be a lot easier.」

—— This summarizes the core best practice for the new branch management workflow, providing clear guidance for daily development operations.

「Let's first make sure that everybody is comfortable with event sourcing. Everybody first learns the fundamentals, you know, and how to implement it before we start putting it everywhere, right?」

—— This sets a clear, risk-controlled adoption strategy for new technical architecture, preventing premature overuse of unmastered technologies.

👍 智能纪要反馈收集 [该内容不支持导出查看]

(注：部分内容可能由 AI 生成)